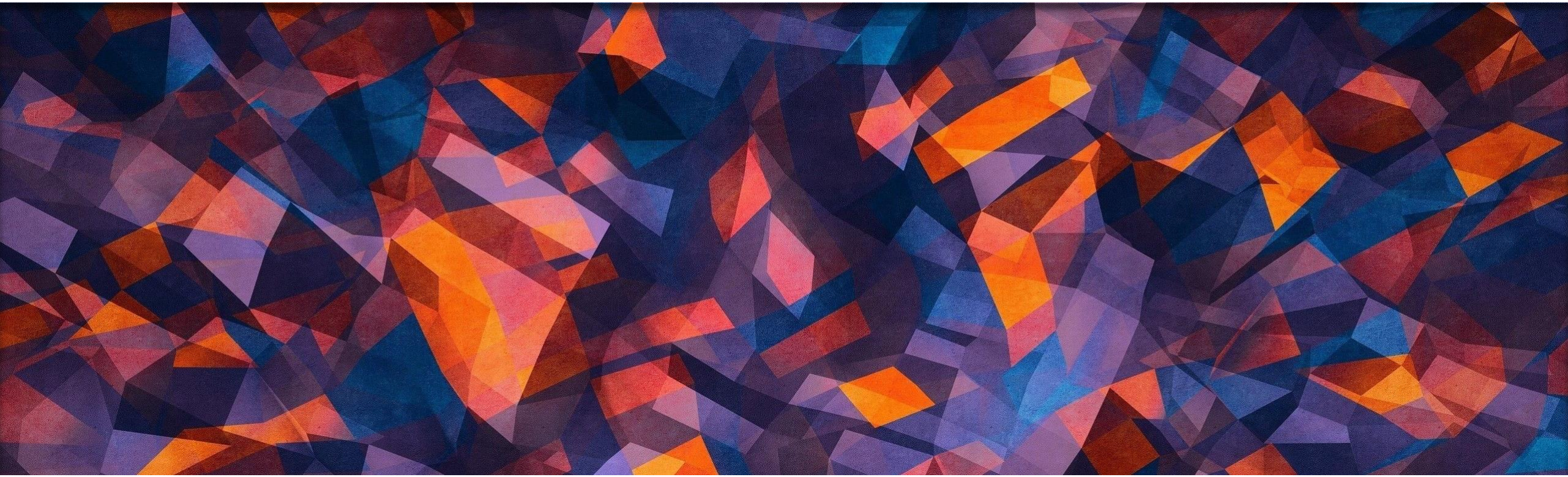
Universität St.Gallen



Event-driven and Process-oriented Architectures

Course Focus and Positioning

Lecture 0

Universität St.Gallen

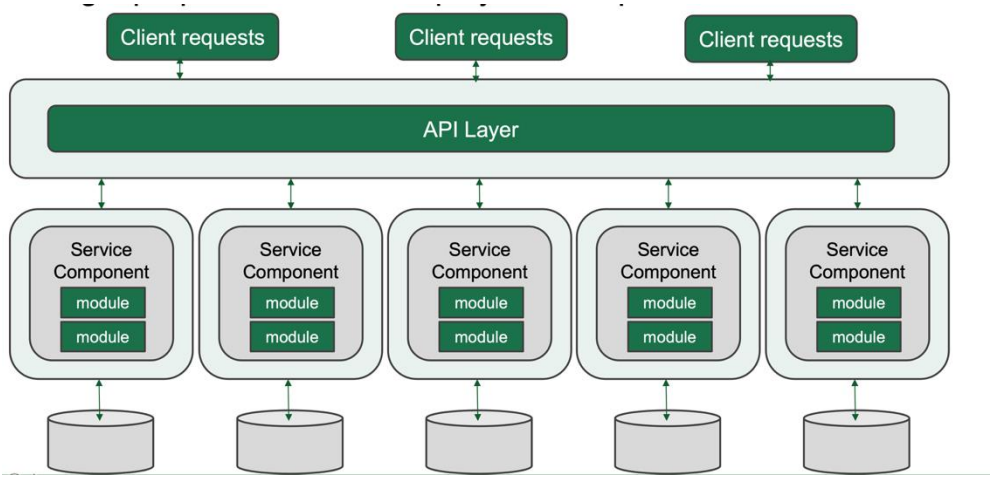
Your Teachers

Prof. Dr. Barbara Weber
Chair for Software Systems Programming and Development

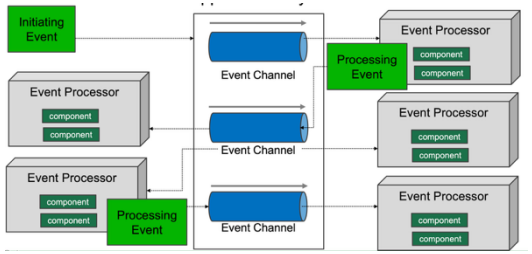
Dr. Amine Abbad-Andaloussi
Senior Researcher

Event-driven and Process-oriented Systems

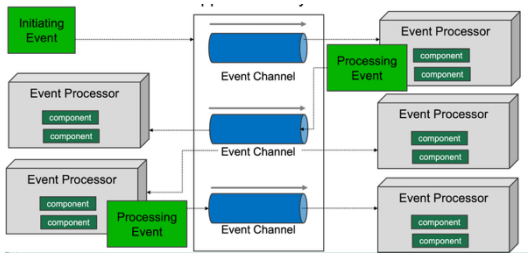
- Advanced Software and Systems Engineering (ASSE)
  - Fundamental of software architecture
  - Microservices architecture

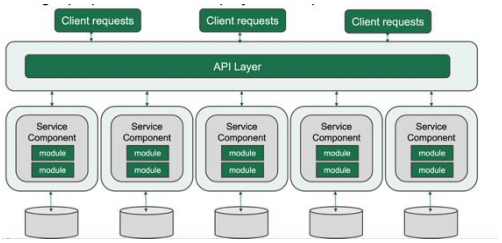


- Focus of this course
  - Event-driven architecture
  - Event-driven microservices



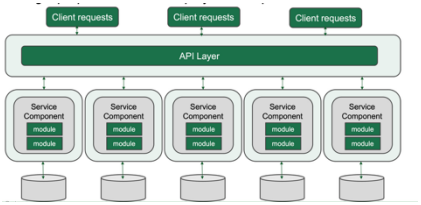
Event-driven architecture





Event-driven Microservices

Universität St.Gallen



Microservices Architecture
Characteristics Rating

Partitioning Type	Domain
Agility	+++++
Deployability	+++++
Elasticity	+++++
Evolvability	+++++
Fault tolerance	+++++
Scalability	+++++
Testability	+++++

Overall cost	+
Performance	++
Simplicity	+
Workflow	+

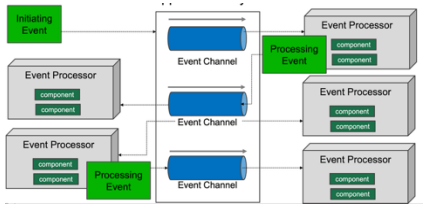
Use when

- Agility (time to market) is critical
- Elasticity, evolvability, fault tolerance, and scalability are important
- Natural fit for cloud development

Do not use when

- When there is a single monolithic database
- When time and budget are major concerns
- When teams are organized in a traditional way (i.e., dedicated UI, backend, database team)
- If services talk too much to each other (since this can bring down many of the high star ratings to 1s or 2s)

Universität St.Gallen



Event-driven Architecture
Characteristics Rating

Abstraction	**++++**
Elasticity	**++++**
Evolvability	**+++++**
Fault tolerance	**+++++**
Performance	**+++++**
Scalability	**+++++**
Workflow	**+++++**

Simplicity	**+**
Testability	**++**

Use when

- When performance is critical
- When elasticity, evolvability, fault tolerance, and scalability matter
- If most of the processing is about responding to things that happen

Do not use when

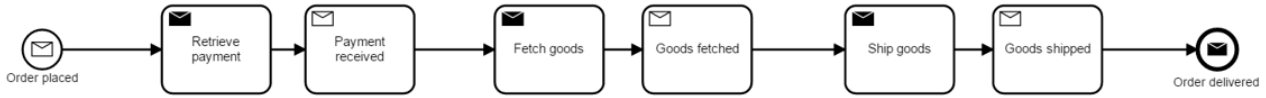
- you need ACID transactions (this style is all about eventual consistency)
- you have a lot of synchronous workflow
- you need to control the timing of events

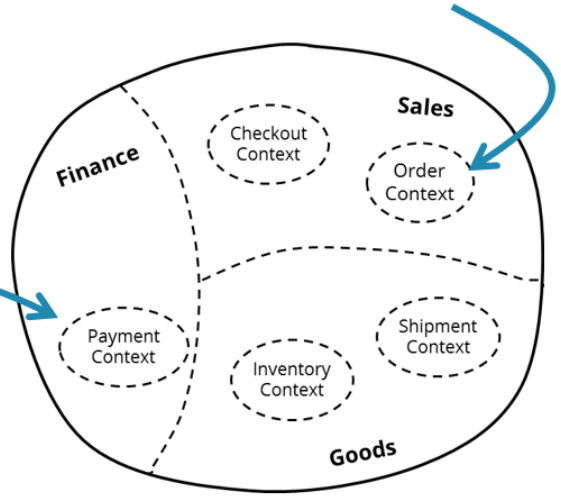
Course: Event-driven and Process-oriented Systems

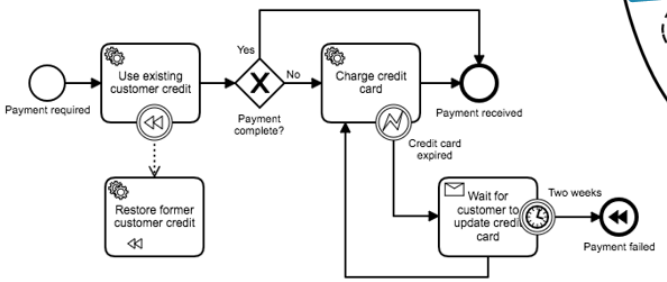
Project Part 1 — Event-driven Microservices & Distributed Transactions

In Part 1, you will learn to:

- Design **event-driven microservices** and message flows
- Apply **orchestration and choreography** patterns
- Implement **distributed transactions** (e.g., sagas, compensation)
- Model domain boundaries and event interactions
- Understand how processes and events integrate in modern architectures



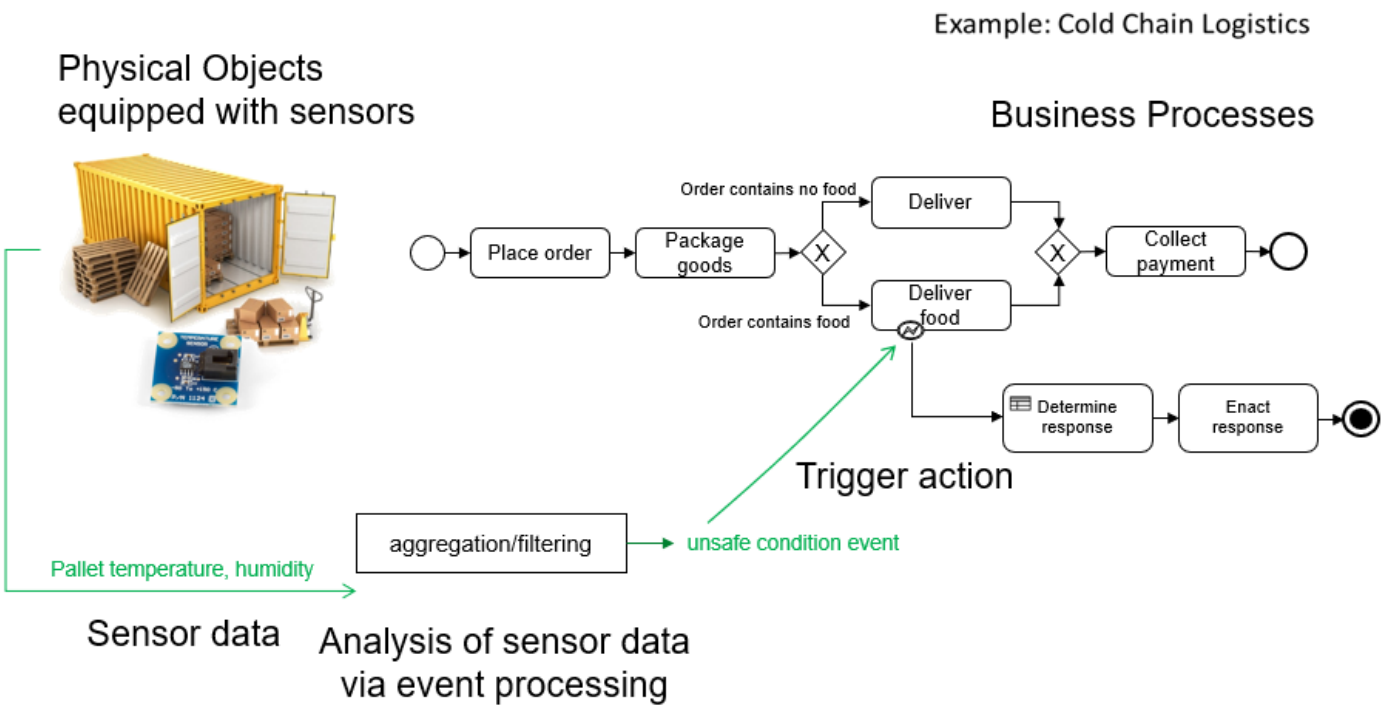




https://github.com/berndruecker/flowing-retail

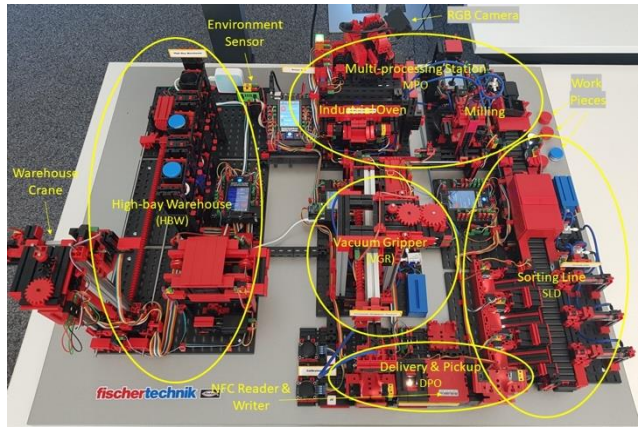
Course: Event-driven and Process-oriented Architectures

- **Project Part 2 — Event Stream Processing & Real-time Event Detection**
- **In Part 2, you will learn to:**
  - Process and analyze **real-time event streams** from IoT devices
  - Identify **business-relevant** conditions from data
  - Combine **event stream processing** with process automation
  - Trigger downstream actions based on meaningful events
  - Build streaming applications using tools such as **Kafka Streams / ksqlDB**

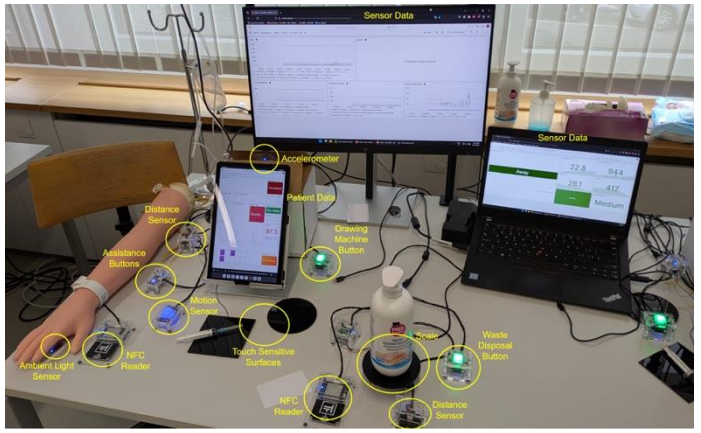


Course: Event-driven and Process-oriented Architectures

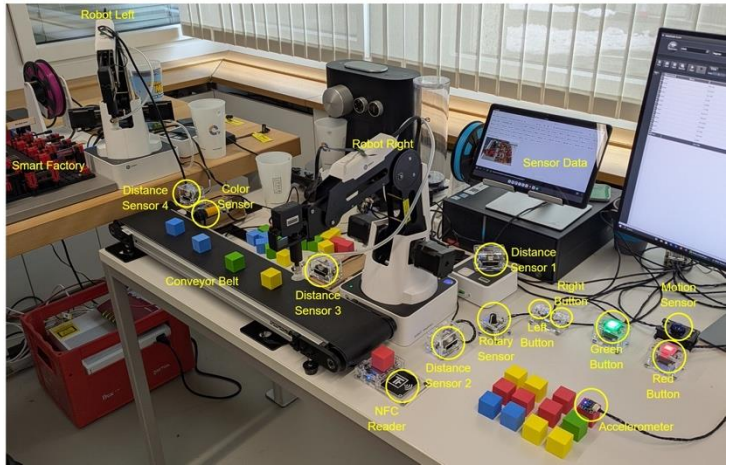
- Lab Infrastructure: Hands-on access to real cyber-physical systems



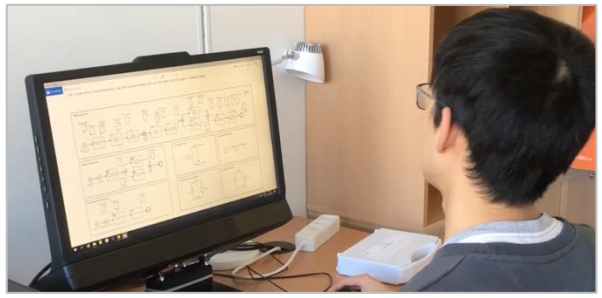
Smart Factory with REST API and MQTT streaming



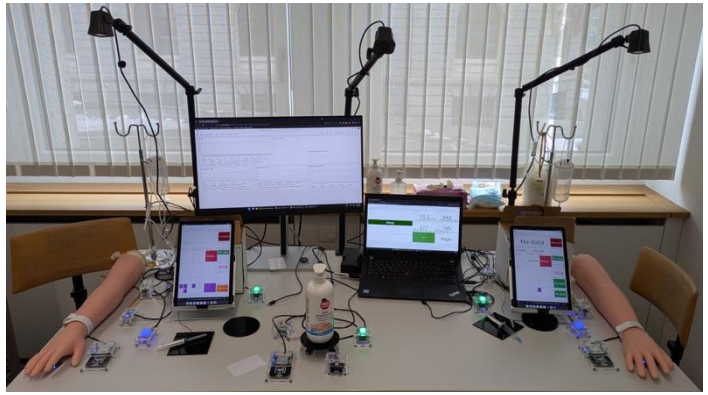
Smart healthcare with sensors (MQTT) and cameras for activity monitoring



Robot arms & conveyor belt with REST API, sensors with MQTT streaming



Eye-tracking



Learning Objectives

- **Learning Objective 1:** Gain the foundational understanding needed to critically evaluate when to use which architectural paradigm in modern distributed systems.
- **Learning Objective 2:** Acquire practical skills to design systems that combine the responsiveness of EDA with the control and transparency of process engines.
- **Learning Objective 3:** Develop hands-on expertise in event streaming technologies that enable real-time processing pipelines.
- **Learning Objective 4:** Learn how to handle high-volume, high-velocity event flows and build systems capable of real-time analytics and reaction.
- **Learning Objective 5:** Understand how real-time events feed into larger architectures and how high-level event streams can be used to gain operational and system-level insights, making these skills highly relevant for both engineering and data-driven organizations.

Course Content

- Lecture
  - Focus on key concepts and backgrounds
- Labs (ungraded, done during class, guided)
  - Focus on the practical application of lecture content
  - Introduction of respective technologies
- Software project (done in groups of max 3, graded individually)
  - Goal is to apply the content covered in lecture and/or lab to projects chosen by your team

Software Project

- Goal is to apply the content covered in lecture and/or lab to projects within an application domain chosen by your team
- Per hand-in choice of a) a larger software application integrating the different topics (Part 1 and 2 of the lecture) or b) two smaller software projects with software applications covering each part of the lecture individually
- Each week we will make suggestions of how to cover the weekly topics, but you will have the freedom to make your own choices within certain boundaries
- Strong recommendation: Align project ideas with teacher
- Expectation: All topics should be covered by the hand-in

Universität St.Gallen

Example of a portfolio-ready student project (built in EDPO 2025)

Combining Events, Microservices and Processes for Automation

- Project developed by 3 students in EDPO 2025
- Sorting blocks by color and presence of NFC tag
- Orchestrated by Camunda 7, modeled in BPMN 2.0, support by MQTT, Kafka and Kafka Streams
- Microservices in Java, Python, C#, Go, Kotlin

Grading

- 2 Projects, done in groups, graded individually
- Grading of software applications, project reports (individual contributions), presentations
  - Hand-in 1 (50%) – Focus event-driven architecture, process automation and orchestration
    - Presentation 16.04.2026
    - Hand-in: 16.04.2026 (Exercises 1-5)
    - Intermediate hand-ins: 03.03.2026 (Exercises 1&2) and 17.03.2026 (Exercises 3&4)

  - Hand-in 2 (50%) – Focus stream processing
    - Presentation 21.05.2026 (Exercises 6-8)
    - Hand-in: 01.06.2026 (Exercises 1-8)

Project Hand-ins

- Git repository with your software applications
- A clear indication of who in the team was responsible for which parts
- A description of your projects including
  - explicit references to the concepts of the lecture and how they are covered by the project
  - a discussion of trade-offs and important design decisions whenever relevant
  - reflections and lessons learnt
  - diagrams explaining your implementation at a conceptual level

Data Sets & Streams from Smart Factory

- Can be used in your projects

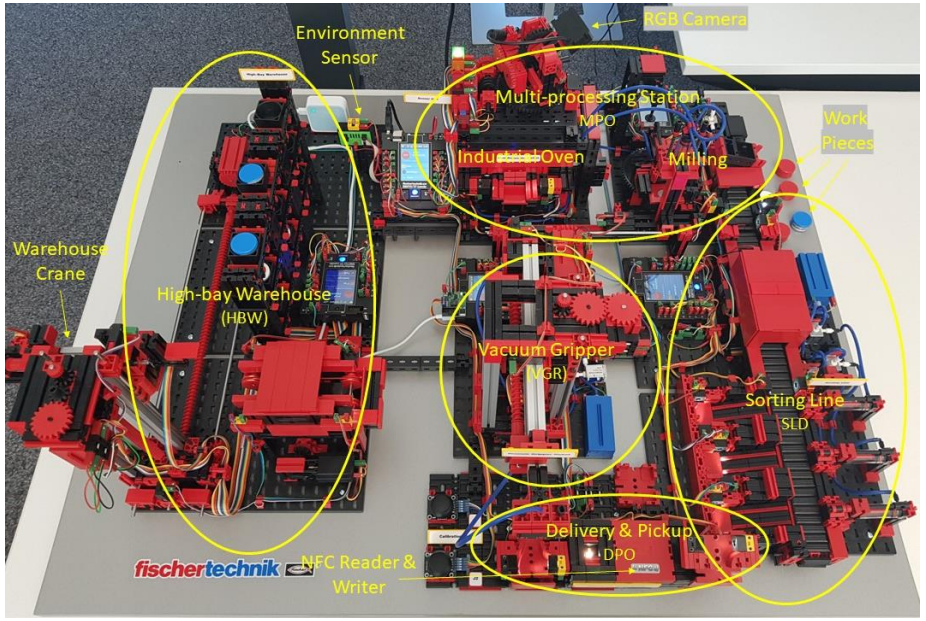
High-level Data Set

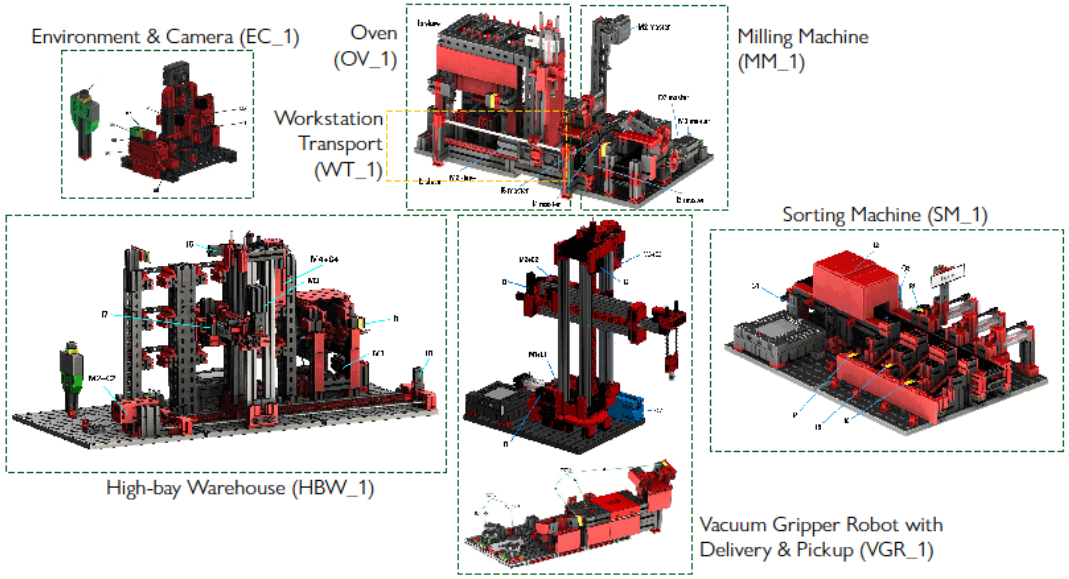
- Status of all stations + production
- Camera
- NFC, environment
- https://zenodo.org/record/7610985
- Live streaming and simple control via MQTT

Low-level Data Set

- Status of all sensors and actuators from all stations
- Data about activities and processes
- https://zenodo.org/record/7612698
- Live streaming via MQTT

- REST API to control smart factory (Python Flask)

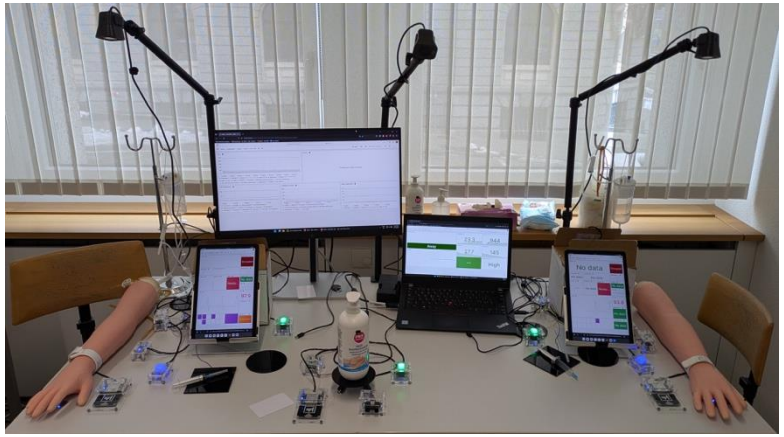


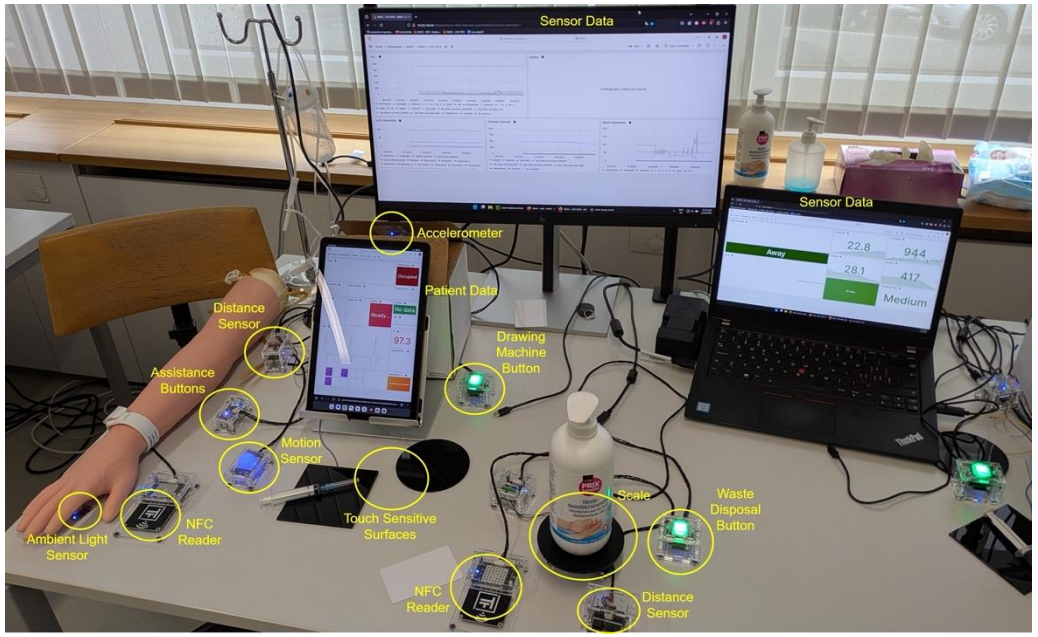


Data Streams from Smart Healthcare

- Can be used in your projects
  - Interface to sensor streams available via MQTT
  - Dataset: https://zenodo.org/records/14860395

  2 donor-, 1 hand hygiene- station:
    - NFC readers + tags
    - Distance and motion sensors
    - Ambient light sensors
    - Multi-touch sensors
    - Accelerometers
    - Load cell (scale)
    - Buttons
    - 3 cameras optionally available
    - Tablets for visualizing dashboards, feedback





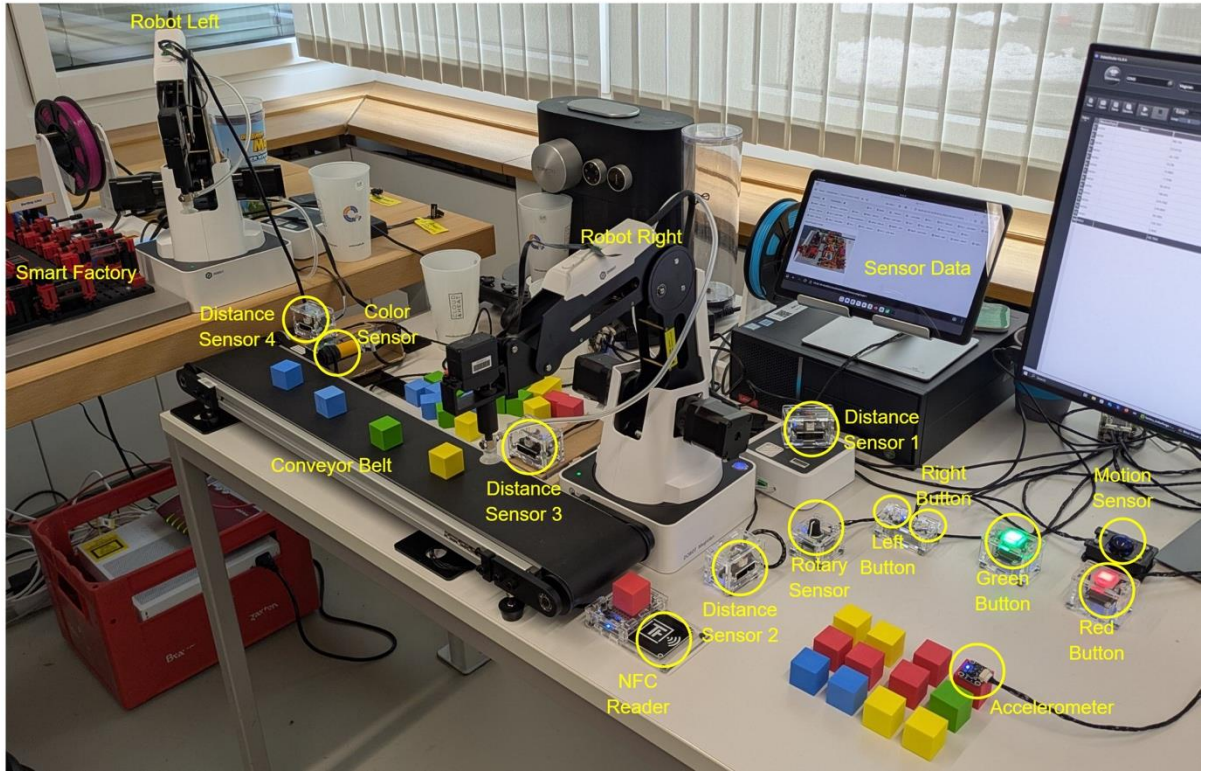
Data Streams from Robots Station

- Can be used in your projects
  - Interface to sensor streams available via MQTT (Python)
  - Interface to control robots + conveyor via REST API (Python Flask, work in progress)

2 robot arms (Dobot Magician)

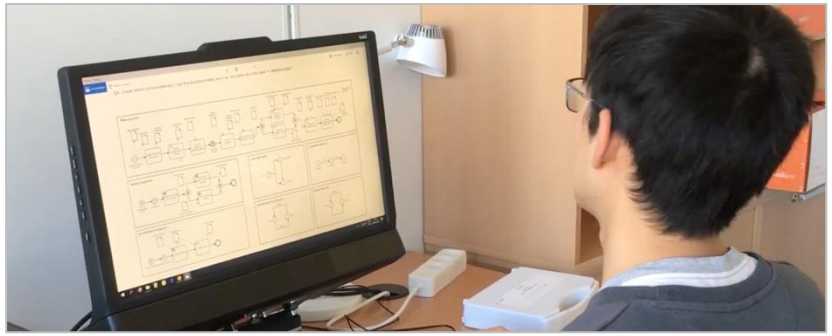
+ Conveyor belt

- With additional sensors:
  - Color
  - Distance, light barrier
  - NFC
  - Motion
  - Rotari poti
  - Accelerometer
  - Environment
  - Buttons

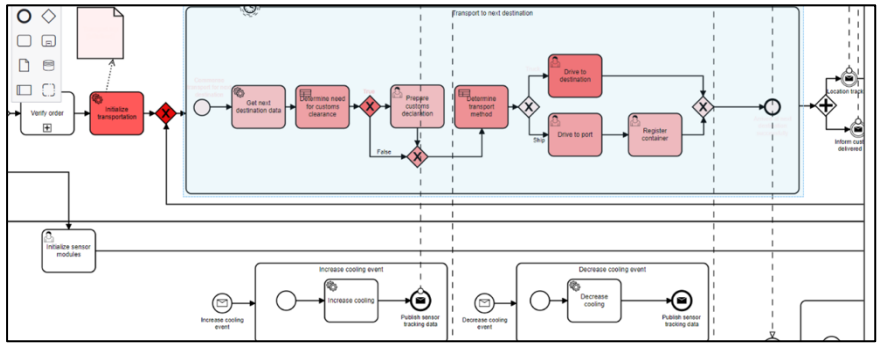


Data Streams and Data Sets for Eye-Tracking Studies

- Can be used in your projects to develop monitoring and adaptive systems
  - **Option 1:** Real streams of mouse-click and eye-tracking data captured using a high-end eye-tracker available in our lab
  - **Option 2:** Simulated streams of mouse-click and eye-tracking data, reflecting users‘ engagement with software artifacts
    (e.g., source-code, process models), examples will be shown in the lab sessions
  - **Option 3:** Eye-tracking datasets collected in the context of different experiments investigating software artifacts
    - Can be replayed and used in event-driven applications



(1) Collection/Streaming of gaze data



(2) Online analysis of users’ cognitive, behavioral and attentional processes

Universität St.Gallen

Date	Content
Week 1: 19.02.2026	The Distributed Streaming Platform Kafka
Week 2: 26.02.2026	Event-driven Architecture
Week 3: 05.03.2026	Process Automation and Process Orchestration: Part 1
Week 4: 12.03.2026	Process Automation and Process Orchestration: Part 2
Week 5: 19.03.2026	Integration Challenges and Transactional Sagas
Week 6: 26.03.2026	Guest Lecture (tentative)
Break	
Week 7: 16.04.2026	**Project presentations**
Week 8: 23.04.2026	Stream Processing 1
Week 9: 30.04.2026	Stream Processing 2
Week 10: 07.05.2026	Stream Processing 3
Week 11: 21.05.2026	**Project presentations** Research Outlook

Course: Event-driven and Process-oriented Architectures

- Industry-relevant Technologies

spring boot

APACHE kafka®

zeebe by Camunda

camunda

OPERATON

docker